

ESTACIÓN 6 · COHORTE 2

Implementando

Scaffolding, diseño agencial y sistemas de ejecución

Primero generamos este deck

01

Leer `PRODUCT.md`.

02

Leer `DESIGN.md`.

03

Usar este storyboard Markdown.

04

Generar slides HTML.

05

Renderizar PDF.

06

Usar el resultado durante la sesión.

SIGUE DEMO: contexto canónico → HTML → PDF → clase

DESIGN STANDARDS AND SKILLS

Skills dan hábitos. DESIGN.md da memoria.

Estándares y skills de diseño según el artículo "Fixing Visual AI Slop".

El agente necesita hábitos instalados

Impeccable

Make Interfaces Feel Better

Web Design Guidelines

Userinterface Wiki

Accessibility · Best Practices · Core Web Vitals · Performance · SEO

React Best Practices

Imagegen

Google DESIGN.md

Criterio y memoria visual viven en archivos

PRODUCT.md	Audiencia, propósito, tono y promesa.
DESIGN.md	Tokens, componentes, layout y anti-patronos.
Linting	Estructura revisable antes de producir artefactos.
Skills	Revisión y ejecución con mejores defaults.

El agente empieza con contexto verificable

- PRD o ISB con problema definido.
- Artefactos de Inception.
- Unidad o tareas priorizadas.
- Diseño de Construction o contrato suficiente.
- Repo o sandbox listo para ejecución.

Key point: Contexto explícito convierte una tarea en trabajo ejecutable.

La pregunta central: qué sistema permite producir

Spec	Contrato de entrada.
Arnés	Superficie de ejecución.
Orquestación	Coordinación de alcance, contexto y trabajo.
Evidencia	Cierre verificable del trabajo.

Antes de ejecutar, prepara el terreno

Unidad o feature

Tareas pequeñas

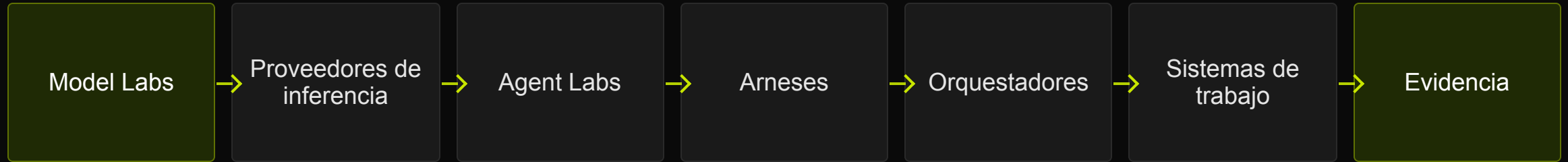
Criterios de aceptación

Comandos de validación

Contexto operativo

Evidencia esperada

Modelos, inferencia, arneses, orquestación



Separar capas que suelen mezclarse en conversaciones de herramientas.

El modelo marca capacidades

Razonamiento

Código

Seguimiento de instrucciones

Multimodalidad

Contexto largo

Latencia percibida

Inferencia convierte capacidad en operación

Ventana de contexto

Cache tokens

Tool calling

Multimodalidad

Throughput y latencia

Costo operativo

Límites de API

Benchmarks ayudan a calibrar expectativas

- Terminal Bench.
- Artificial Analysis.
- Benchmarks de código y razonamiento.
- Pruebas reales sobre tu repo.
- Costo y latencia en contexto operativo.

Key point: Un benchmark orienta; la evidencia de tu repo decide.

El arnés es donde el agente toca el mundo

Lee archivos

Ejecuta comandos

Edita código

Usa permisos

Observa resultados

Devuelve evidencia

Superficies distintas, trabajo verificable

Claude Code

Codex

OpenHands

Factory

OpenCode

Antigravity 2

Pi

Presentar características observables: superficie, permisos, contexto, ejecución, validación y evidencia.

Seis piezas mínimas



El backlog también es contexto

Beads

Linear

GitHub Issues

Markdown local

Issues como unidades de ejecución

PRs como evidencia de cambio

Preparar el puente hacia Estación 7.

Orquestar coordina alcance, contexto y ejecución

Divide trabajo

Controla dependencias

Asigna agentes o arneses

Recoge evidencia

Coordina validación

Mantiene continuidad

La coordinación crece con el sistema

Muchas tareas dependientes

Varios agentes o ramas

Validación como bloqueo

Handoffs frecuentes

Evidencia dispersa

Repetición suficiente para automatizar

Un orquestador organiza trabajo, agentes y evidencia

Tareas con schema

Dependencias

Checkpoints

Estados

Validaciones

Handoffs

Prepara tu scaffolding operativo

- ☐ Ficha de arnés.
- ☐ `AGENTS.md` o equivalente.
- ☐ Una skill de proyecto.
- ☐ `PRODUCT.md` y `DESIGN.md` si hay interfaz.
- ☐ Validaciones disponibles.
- ☐ Nota de orquestación futura.

La próxima estación convierte AI-DLC en trabajo orquestable.